

Drydock Quick Start

Build your first project as a small, reviewable slice. Drydock turns written project intent into a Blueprint, turns the Blueprint into a dependency-aware Manifest, and builds the software one verified unit at a time.

This guide assumes a greenfield project and a Unix-like shell. Replace **MyApp** with your Target name. A Target is Drydock's workspace for one project; it is not the generated application.

The core rule

The Blueprint is the authority. The Manifest is the execution plan. The generated application is the result.

When the product changes, update the Blueprint first, run **drydock refit**, and then build again. Do not make an undocumented code change and expect the next build to understand it.

Before you start

Install Python 3.11 or later, Drydock, and one subscription-authenticated provider CLI:

```
uv tool install drydock-sdd
drydock --version
```

Install and sign in to either **claude** or **codex**, then select it for Drydock:

```
drydock config set llm_provider claude
# Or:
# drydock config set llm_provider codex
```

Create and configure a workspace. The workspace stores Targets and their planning and verification artifacts. The build directory stores generated applications.

```
mkdir -p "$HOME/drydock"
drydock config set drydock_workspace "$HOME/drydock"
drydock config set drydock_build_directory "$HOME/drydock/build"
drydock config show
```

1. Start with a small project

Your first Target should be small enough to explain in a few paragraphs and verify in one sitting. A command-line tool, a small HTTP service, or a focused data utility is a good first project.

Good first-project boundaries:

- one primary user or operator;

- one or two core workflows;
- a small, explicit input and output surface;
- a testable definition of done;
- no requirement to solve deployment, billing, or every future integration immediately.

Create the Target:

```
drydock init MyApp \  
  --display-name "My App" \  
  --description "A small working software product."  
  
drydock status MyApp
```

The Target is created at `$DRYDOCK_WORKSPACE/targets/MyApp/`. The generated application will be created below `$DRYDOCK_BUILD_DIRECTORY/MyApp/`.

2. Write useful source material

Put the initial project description in one or more Markdown files outside the Target. Keep the source specific enough to build, but short enough to review. Avoid turning the first source file into a complete architecture document.

A useful starting document answers:

1. Who uses the product?
2. What problem does it solve?
3. What is the smallest successful workflow?
4. What inputs does the workflow accept?
5. What outputs or observable effects does it produce?
6. What must happen when the input is invalid or unavailable?
7. How will you know the workflow works?

For each important capability, name its interface points. Use routes for a web application, commands for a CLI, public symbols for a library, datasets or files for a pipeline, and topics or events for an event-driven system. These names help Drydock derive dependencies between stories.

Example source material:

```
# Reading List  
  
## User  
  
A reader maintains a personal list of books to read.  
  
## First workflow  
  
The reader adds a book by title and author. The application rejects an empty  
title, stores valid  
books, and lists saved books in insertion order.
```

Interface

- CLI command: `reading-list add --title TITLE --author AUTHOR`
- CLI command: `reading-list list`

Done when

- A valid book can be added and appears in `list` output.
- Empty titles are rejected with a non-zero exit status.
- The application has automated tests for both cases.

Import the source into the Target. Imported files are retained under `blueprint/sources/` as read-only planning input.

```
drydock import MyApp ./notes --format markdown
```

Best practice: keep source files focused by concern. Use separate files for product intent, constraints, and operational requirements rather than one large document containing unresolved alternatives.

3. Analyze, answer, and approve

Analyze the imported material before asking Drydock to create the Blueprint:

```
drydock analyze MyApp
```

Review these artifacts in the Target:

Artifact	Review purpose
<code>ANALYSIS.md</code>	Derived features, stories, scope, and recommendations
<code>BLOCKERS.md</code>	Decisions that prevent safe planning; resolve these before planning
<code>SEA_TRIALS.md</code>	Project-level acceptance and release objectives
<code>COMPASS.md</code>	Durable project guidance used by later commands
<code>QuarterDeck/</code>	Persistent questionnaires and review state

If `BLOCKERS.md` exists, answer or resolve the blockers and run `drydock analyze MyApp` again. Use the QuarterDeck when you want a browser-based review surface:

```
drydock run quarterdeck MyApp
```

Do not treat an LLM-generated analysis as approval. The Commander reviews scope, corrects wrong assumptions, answers material questions, and confirms the definition of done.

4. Create and inspect the Blueprint

Once analysis is ready, create the typed specifications and Manifest:

```
drydock plan MyApp
```

Inspect the results before building:

```
find "$HOME/drydock/targets/MyApp/blueprint" -maxdepth 1 -type f -print  
sed -n '1,220p' "$HOME/drydock/targets/MyApp/MANIFEST.md"  
drydock build status MyApp
```

The Blueprint should be understandable without reading generated code. Check that:

- each story has a clear outcome and acceptance criteria;
- stories are small enough to build and test independently;
- **Provides**, **Consumes**, and **Depends On** describe real interfaces;
- negative paths and invalid input are covered;
- the Manifest order reflects actual dependencies;
- project-wide goals appear in **SEA_TRIALS.md**, not only in a story's prose.

If the plan is wrong, correct the source or Compass guidance and rerun the appropriate planning step. Do not begin by manually editing generated planning output unless the command contract explicitly assigns that file to the Commander.

5. Build the first slice

Preview the next build when you want to inspect its scope:

```
drydock build MyApp --dry-run --show-prompt
```

Build the runnable frontier:

```
drydock build MyApp  
drydock build status MyApp
```

Build iteratively. After each build, inspect the generated application and its tests, then resolve failures before adding unrelated scope. Use a narrower selector when only one unit needs attention:

```
drydock build MyApp --step <step-id>
# Or:
# drydock build MyApp --story <story-id>
```

The build process records prompts, raw provider output, command output, and build evidence in the Target logs. Keep those artifacts with the Target when reviewing or diagnosing a build.

6. Verify before calling it done

Run deterministic acceptance checks and the project-level release review:

```
drydock score ac MyApp
drydock score release MyApp
```

Review [SOUNDINGS.md](#) for deterministic acceptance results, [SCORECARD.md](#) for the release assessment, [SEA_TRIALS.md](#) for project-level objectives, and `drydock build status MyApp` for unfinished or blocked work.

Do not call a project complete because the build command exited successfully. Completion requires the intended acceptance criteria to pass and the remaining Manifest work to be closed or deliberately deferred.

7. Change the project safely

Use the same sequence for the next feature:

1. Update the source of truth or create a change ticket.
2. Run `drydock refit MyApp`.
3. Review the affected Blueprint and Manifest work.
4. Build the affected frontier.
5. Run acceptance and release scoring again.

```
drydock refit MyApp
drydock build status MyApp
drydock build MyApp
drydock score ac MyApp
```

If the design is not settled, use the `/refit` skill to capture the discussion in the Target before applying the approved change.

A practical first-session checklist

- ☐ Drydock is installed and `drydock --version` works.
- ☐ `claude` or `codex` is installed, authenticated, and selected in Drydock.
- ☐ The workspace and build directory are configured and understood.
- ☐ The first Target has one narrow, testable outcome.

- ☐ Source material names users, workflows, interfaces, failures, and done criteria.
- ☐ `drydock analyze` has no unresolved blockers.
- ☐ A human has reviewed the analysis and answered material questions.
- ☐ The Blueprint and Manifest describe the intended scope and dependency order.
- ☐ The first build has been previewed or reviewed before execution.
- ☐ Acceptance and release scoring have been run.
- ☐ The next change will update the Blueprint before code is rebuilt.

Common mistakes

Mistake	Better practice
Starting with a large, vague product brief	Build one narrow workflow first
Skipping analysis review	Resolve blockers and approve scope before <code>plan</code>
Treating generated code as the specification	Keep the Blueprint authoritative
Packing multiple unrelated features into one story	Split stories by independently verifiable outcome
Describing only the happy path	Specify invalid input, missing data, and failure behavior
Running every command from the Drydock repository	Configure the workspace; commands resolve Targets there
Editing code without updating the Blueprint	Run the refit workflow so the plan and software stay aligned

For complete command contracts and architecture, see the [Drydock Specification](#). For installation details, see the [User Installation Guide](#).